

Linguaggi di Programmazione

Modulo II

Nicolò Giuliani

Indice

1	Nomi e ambiente	2
1.1	Regole di scope	3
2	Memoria	4
2.1	Implementazione delle regole di scope	6
3	Controllo	7
3.1	Espressioni	7
3.2	Comandi	7
3.2.1	Iterazione	9
3.3	Ricorsione	9
4	Sottoprogrammi	10
4.1	Parametri	10
4.2	Downward funarg problem	11
4.3	Upward funarg problem	12
4.4	Eccezioni	13
5	Introduzione ai Tipi	14
6	Tipi Base e l'Algebra dei Tipi	15
7	Tipi Polimorfi	18
8	Exceptions	19
9	Sicurezza della memoria: Garbage collection e Borrow-checking	20
9.1	Tombstones	20
9.2	Locks and Keys	20
9.3	Garbage Collection	21
9.3.1	Reference Count	21
9.3.2	Mark and Sweep	22
9.3.3	Stop and Copy	23
9.3.4	Borrow checking	23
9.4	Lifetimes	24
10	Tipi di Dato Astratto e Object Orientation	24

1 Nomi e ambiente

L'uso di un **nome** serve ad indicare l'oggetto denotato.

```
const pi = 3.14;
int x;
void f() {...};
```

π , x , f sono nomi. Gli oggetti che denotano:

- la costante 3.14
- una variabile
- la definizione di f

Nei linguaggi i nomi sono **token** (identificatori). Il loro utilizzo serve per indicare l'**oggetto denotato**.

Definiamo **ambiente** ovvero l'insieme delle associazioni fra nomi e oggetti denotabile esistenti a run-time in uno specifico punto del programma ed in uno specifico momento dell'esecuzione. Attraverso la **dichiarazione** (meccanismo col quale si) si crea un'associazione in ambiente.

```
int x;
int f(){
    return 0;
}
type T = int;
```

Bisogna stare attenti come in diversi punti del programma lo stesso nome può essere utilizzato per oggetti diversi. Mentre attraverso **aliasing** possiamo avere nomi diversi per lo stesso oggetto (puntatori per riferimento, puntatori)

```
int *x;
x = (int *) malloc(sizeof(int))
*x = 5;
```

Nei linguaggi moderni l'ambiente è strutturato attraverso i **blocchi**. Ovvero regioni testuali con inizio e fine che può contenere dichiarazioni locali in quella regione.

```
{
int tmp = x;
x = y;
y = tmp;
}
```

Due blocchi si sovrappongono se un blocco sta dentro un altro blocco. La dichiarazione locale è libera per un blocco annidato, a meno che una nuova dichiarazione venga fatta nel sottoblocco. L'ambiente locale può essere suddiviso in

- ambiente locale: associazioni all'ingresso del blocco
- ambiente non locale: associazioni eritate da altri blocchi
- ambiente globale: parti di ambiente non locale relativo alle associazioni comuni (variabili globali, ecc)

La **vita** di un oggetto e un legame è:

- vita dell'oggetto più lunga

```
fn P(int X){...} //definiamo la funzione
int A; //definiamo la variabile
P(A); //creiamo il legame
```

Durante l'esecuzione di $P(A)$ esiste un legame tra X e l'oggetto che esisteva già prima e dopo l'esecuzione della funzione.

- vita dell'oggetto più breve (area di memoria de-allocata)

```
int *x, *y;
x = (int *) malloc(sizeof(int));
y = x;
free(x);
x=null;
```

Dopo la $free(x)$ non esiste più l'oggetto ma esiste ancora un legame per y (dangling reference)

1.1 Regole di scope

Come implementiamo la regola di visibilità?

Una dichiarazione locale è visibile in quel blocco e in tutti i blocchi annidati, a meno di nuove dichiarazioni con stesso nome.

```
{int x=10;
  void foo () {
    x++;
  }
  void fie () {
    int x = 0;
    foo();
  }
  fie();
}
```

Quale x incrementa $foo()$? Siamo in un caso di un riferimento non-locale, può essere risolto:

- Scope statico (c, java, rust, ecc): nel blocco che include sintatticamente $foo()$

$$x = 11$$

- Scope dinamico (Emacs Lisp, perl, ecc): nel blocco che è eseguito immediatamente prima di $foo()$

$$\begin{aligned} x_{\text{locale}} &= 1 \\ x &= 10 \end{aligned}$$

Scope statico

```
{ int x=0;
void pippo(int n){
    x = n+1;
}
pippo(3);
// incrementa x globale
write(x);
// stampa x globale = 4
{int x=0;
// ridifiniamo x locale
pippo(3);
// incrementa x globale
write(x);
// stampa x locale = 0
}
write(x);
// stampa x globale = 4
}
```

Output: 404

Scope dinamico

```
{ int x=0;
void pippo(int n){
    x = n+1;
}
pippo(3);
// incrementa x globale
write(x);
// stampa x globale = 4
{int x=0;
// settiamo x locale
pippo(3);
// incrementa x di blocco chiamante
write(x);
// stampa x locale = 4
}
write(x);
// stampa x globale = 4
}
```

Output: 444

Per l'allocazione di memoria guarda: Implementazione delle regole di scope.

2 Memoria

Come allochiamo la memoria?

- Statica: allocazione a tempo di compilazione.
Ovvero zone protette di memoria dove gli oggetti hanno indirizzi assoluti per tutta l'esecuzione. (variabili globali, costanti, ecc). Non permette la ricorsione, perche tiene SOLO le variabili locali per sottoprogrammi).

```
fn fatt(x){
    if x == 0 {
        return 1
    }
    else return x*fatt(x-1)
}
fatt(5):
    5*fatt(4)
fatt(4):
    4*fatt(3)    \\ sovrascrive il vecchio x
fatt(3):
    3*fatt(2)    \\ sovrascrive ancora
```

Quindi sovrascrivendo sempre x non possiamo mai avere un valore di ritorno valido per la ricorsione.

- Dinamica: allocazione a tempo di esecuzione.

- **Stack** (FIFO): Definiamo che ogni istanza di un sottoprogramma ha una porzione di memoria chiamato **frame stack** (record di attivazione o RdA), con le relative informazioni (anche indirizzo di ritorno di memoria).

Abbiamo un puntatore **SP** (stack pointer) che punta sempre il cima allo stack, e poi abbiamo anche **FP** (frame pointer) che indica la base del frame. Così possiamo accedere a tutte le variabili con

$$\text{indirizzo} = \text{FP} + \text{offset}$$

con FP calcolato a compile-time.

Perchè utilizziamo gli RdA e il link dinamico? Gli RdA non hanno tutti la stessa dimensione, dipende da parametri, variabili locali, ecc, quindi il *pop()* non è così semplice. Allora attraverso il link dinamico abbiamo

	variabili locali		
	parametri		
	indirizzo ritorno		
	link_dinamico		\\ punta al frame precedente
			\\ che ci ha chiamato

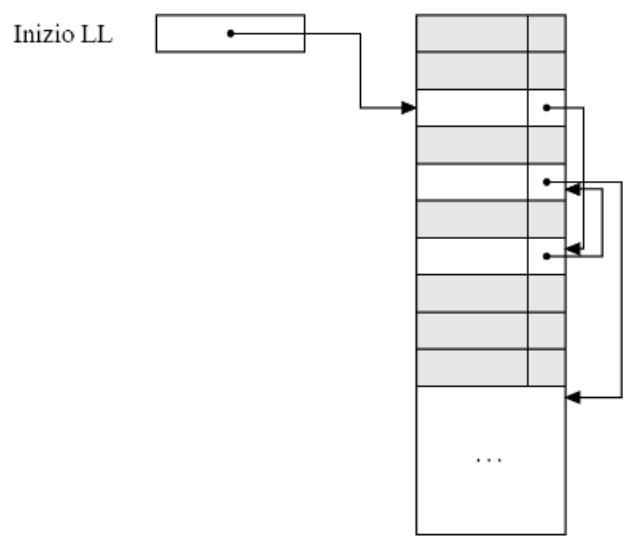
e quindi basta fare $\text{SP} = \text{link_dinamico}$.

- **Heap**: memoria nella quale i (sotto)blocchi possono essere allocati/deallocati dinamicamente arbitrariamente.

Questo ci serve per gli oggetti di dimensione dinamica come stringhe, liste, alberi, puntatori, ecc.

Abbiamo due modi di implementazione:

- Blocchi di memoria di dimensione fissa, li mettiamo nella **LL** (lista libera). Quando allochiamo shiftiamo il puntatore LL al primo blocco libero, mentre quando deallochiamo LL punta a quel blocco che punta al ex-primo blocco libero.



- Blocchi di memoria di dimensione variabile, partiamo con un unico blocco. Quando allochiamo prendiamo una parte del blocco di dimensione richiesta, mentre la deallocazione la restituisce alla LL. Bisogna però stare molto attenti alla frammentazione della memoria!

- * Frammentazione interna: allochiamo un blocco che però è più grande di quello richiesto (spreco di memoria)
- * Frammentazione esterna: abbiamo abbastanza spazi sparsi ma separati

Come gestiamo quindi la LL unica? Ad ogni allocazione possiamo fare:

- first fit: primo blocco abbastanza grande
- best fit: minor blocco abbastanza grande

Possiamo migliorare? Attraverso LL multiple (blocchi dimensione variabile). Però come ripartiamo i blocchi fra le LL?

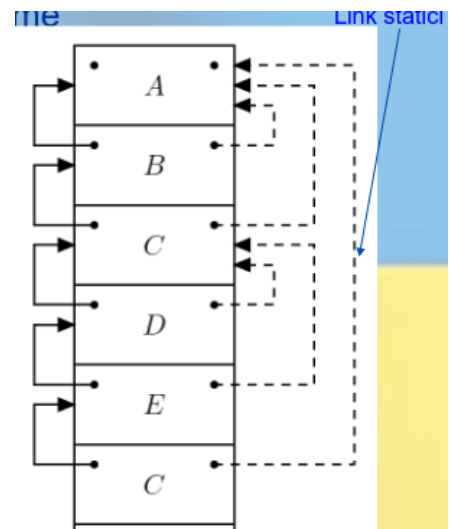
- statica
- dinamica

2.1 Implementazione delle regole di scope

- Scope statico

- Catena statica
Immaginiamo di Avere

main $\rightarrow$ A $\rightarrow$ B $\rightarrow$ C $\rightarrow$ D $\rightarrow$ E $\rightarrow$ C



Se un sottoprogramma è a livello k implica che la catena è lunga k .

Se $k = 0$ il chiamante punta SP alla funzione, se $k > 0$ il chiamante risale la catena k -volte. Ogni volta che incontriamo un nome ci segniamo se è locale con $h = 0$ o se era stato definito h -blocchi fa.

- Display

Possiamo ottimizzare il costo h per la lettura di un nome non locale attraverso un array con i RdA attivo definito ad RdA $i - 1$. Così possiamo trovare la posizione di un oggetto con $i - h$. Quando aumentiamo la catena $k + 1$ il chiamante e il processo non condividono più il $Display[k + 1]$.

Nella realtà il display è più costoso nella sequenza di chiamate e quindi è poco utilizzato.

- Scope dinamico

- A-list
Le associazioni sono memorizzate in una struttura apposita, manipolata come una pila.
- CRT (Tabella centrale dell'ambiente)
Una tabella mantiene tutti i nomi distinti del programma. Se i nomi sono noti il costo è costante, se no costo hash.

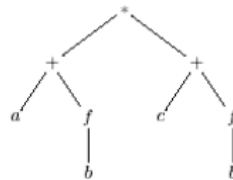
3 Controllo

Come strutturiamo il controllo del flusso di un linguaggio?

3.1 Espressioni

Entità sintattica che produce un valore o è indefinita. La sintassi può essere

- Infissa: $a + b$
Gli operatori aritmetici hanno precedenza su confronto che hanno a loro volta precedenza su logici. Le parentesi sono essenziali in alcuni casi. L'associatività dipende dal tipo di operando.
- Prefissa: $+ab$
Si valuta attraverso una pila, si legge il prossimo simbolo e lo si mette sulla pila, se abbiamo un operatore applichiamo l'operazione su i valori della pila prima del operatore e lo memorizziamo R . Eliminiamo operatore e operandi, memorizziamo R in pila.
- Postfissa: $ab+$
Sempre attraverso una pila ma dobbiamo contare gli operandi che vengono letti



Le espressioni possono essere rappresentate come alberi, a seconda del tipo di visita di albero abbiamo una delle 3 precedenti notazioni lineare. A partire dall'albero il compilatore produce il codice oggetto oppure l'interprete valuta l'espressione.

Ovviamente la valutazione può comportare dei risultati diversi (effetti collaterali)

$(a+f(b)) * (c+f(b))$

Se $f(b)$ modifica b allora il risultato da sinistra a destra è diverso di quello da destra a sinistra.

3.2 Comandi

Un comando è un' entità sintattica la cui valutazione non necessariamente restituisce un valore, ma può avere un effetto collaterale.

Sapendo che una **variabile** è un "contenitore" modificabile di valori con un nome. Attraverso il comando di assegnamento possiamo modificare il valore di una variabile.

```
x := 2
x = x+1
```

Il primo x è un l-value ovvero una locazione, mentre il secondo x è un r-value ossia un valore che può essere contenuto in una locazione. Anche la assegnazione comporta effetti collaterali (anche se non ritorna un valore sempre).

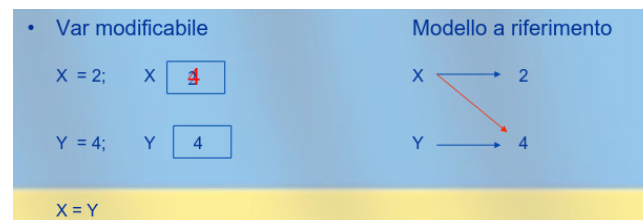
```
//C
x = 2; //ritorna 2
y = x = 2; //valido
```

Questo vale un po' per tutti i linguaggi imperativi. Mentre nei linguaggi funzionali (Haskell, ecc) assomiglia più al modello matematico, una variabile denota un valore ed è immutabile. I linguaggi logici danno la possibilità di modifica entro certi limiti.

Abbiamo anche il **modello a riferimento**, ovvero una variabile è riferimento ad un valore che ha un nome

```
x —> 2
```

Simile ai puntatori, ma i puntatori hanno le allocazioni esplicite.



Ambiente e memoria

: Due variabili diverse possono denotare lo stesso oggetto? Nomi $\rightarrow$ Valori non basta!
 Nei linguaggi imperativi abbiamo

- Valori Denotabili (quelli che possono dare i nomi)
- Valori Memorizzabili (quelli che si possono memorizzare)
- Valori Esprimibili (risultato della valutazione di una exp.)

Come gestiamo i **comandi per il controllo sequenza**?

Esplicitamente con ";", "blocchi" o "goto", ma non bastano per la *programmazione strutturata*. **Comandi condizionali** come "if case" o comandi iterativi "for while". Questi ultimi 2 comandi permettono di avere codice ben strutturato e non "spaghetti code". Questi permettono una maggiore leggibilità e anche ottimizzazione se compilato in modo astuto.

Come sappiamo Iterazione e ricorsione sono i due meccanismi che permettono di ottenere formalismi di calcolo Turing completi. Senza di essi avremmo automi a stati finiti.

3.2.1 Iterazione

Abbiamo due tipologie di iterazione

- indeterminata: cicli controllati logicamente, non noto a priori al avvio del ciclo (while, repeat, ecc). Questo è il vero potere delle MdT. Si implementano attraverso l'istruzione di salto condizionato della macchina fisica.
- determinata: cicli controllati numericamente, ovvero noto a priori (do, for, ecc)

3.3 Ricorsione

Modo alternativo all'iterazione per ottenere il potere espressivo delle MdT.

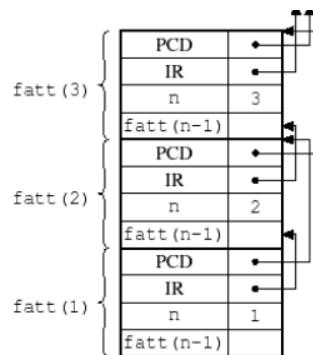
La definizione di una funzione ricorsiva è analoga alla definizione induttiva di una funzione. Ogni programma iterativo può essere riscritto in ricorsivo e viceversa.

In caso di implementazioni naive ricorsione meno efficiente di iterazione possiamo utilizzare optimizing compiler che produce codice efficiente o tail-recursion.

La ricorsione in coda (tail recursion): Una chiamata di g in f si dice "chiamata in coda" (o tail call) se f restituisce il valore restituito da g senza ulteriore computazione.

```
function tail_rec (n: integer): integer
  begin ... ; x:= tail_rec(n-1) end
function non_tail_rec (n: integer): integer
  begin ... ; x:= non_tail_rec(n-1); y:= g(x) end
```

Qui la chiamata ricorsiva è l'ultima operazione eseguita dalla funzione, la funzione ritorna direttamente il risultato della chiamata ricorsiva. Dopo $tail_rec(n-1)$ non c'è più nulla da fare. Non serve allocazione dinamica della memoria con pila: basta un unico RdA.



Possibile la generazione di codice tail-recursive usando continuation passing style (passare il resto come parametro), questo rende più efficiente il costo computazionale e di spazio.

```
int fibrc (int n, int res1, int res2){
  if (n == 0)
    return res2;
  else
    if (n == 1)
      return res2;
    else
      return fibrc(n-1, res2, res1+res2);
}
```

Costo lineare di tempo n e spazio costante (1 RdA).

4 Sottoprogrammi

I LP sono essi stessi astrazioni del calcolatore sottostante.

LP alto livello $\iff$ maggiore astrazione

- Astrazione sul controllo: fornisce astrazione funzionale al progetto (comunica con parametri, ambiente globale)

```
double P (int x) {  
    double z;  
    /* CORPO DELLA FUNZIONE  
    return expr;  
}
```

- Specifica P
- Scrivi P
- Usa P

senza conoscere il contesto

- Astrazione sui dati: implementazione dei dati/operazioni inaccessibile all'utente

4.1 Parametri

```
- dichiarazione/definizione  
int f (int n) {return n+1;}  
  
- uso/chiamata  
x = f(y+3);
```

Parametro formale

Parametro attuale

Come passiamo i parametri?

- per valore: il valore dell'attuale è assegnato al formale, che si comporta come una variabile locale
- per riferimento: è passato un riferimento (indirizzo) all'attuale; i riferimenti al formale sono riferimenti all'attuale (*aliasing*)
- per costante (read-only): nella procedura non è permessa la modifica del formale
- per risultato

```
void foo (result int x) {x = 8;}  
...  
y = 1;  
foo(y);
```

qui y vale 8

- per valore/risultato

```
void foo (value-result int x)  
{ x = x+1; }  
...  
y = 8;  
foo(y);
```

qui y vale 9

- per nome: una chiamata di P è la stessa cosa di eseguire il corpo di P dopo aver sostituito i parametri attuali al posto dei parametri formali

```

int y;
void fie (name int x){
    int y;
    x = x + 1; y = 0;
}
...
y = 1;
fie(y);

```

```

{ int y;
y = y+1;
y = 0; }

```

Può portare conflitti di nomi!

Alcune funzioni permettono di passare come parametro altre funzioni (puntatore al RdA all'interno della pila). O magari restituire delle funzioni (mantenere il RdA della funzione restituita, la pila non va più bene). Come gestire l'ambiente della funzione?

```

int x=4; int z=0;
int f (int y){
    return x*y;}
void g ( int h(int n) ) {
    int x;
    x = 7;
    z = h(3) + x ;
    end;}
...
{ int x = 5;
  g(f);
}

```

Quale x utilizziamo in f ?

- scope statico: la x esterna
- scope dinamico: ha senso sia la x del blocco di chiamata che la x interna

4.2 Downward funarg problem

Quando una procedura viene passata come parametro, si crea un riferimento tra un nome (par formale: h) e una procedura (par attuale: f).

Ovvero: Quale ambiente non locale si applica al momento dell'esecuzione di f (in quanto chiamata via h)?

- **Deep binding**: ambiente al momento della creazione del legame $h \rightarrow f$ (scope statico)
- **Shallow binding**: ambiente al momento della chiamata di f via h (forse scope dinamico)

```

int x=4; int z =0;
int f (int y){
    return x*y; }
void g (int h(int n)){
    int x ;

    x = 7;
    z = h(3) + x
}

...
{int x = 5;
  g(f);
}

```

- Scope statico:
 - Deep: x rossa (esterna)
 - Shallow: x rossa (esterna)
- Scope dinamico:
 - Deep: x azzurra (blocco di chiamata)
 - Shallow: x nera (blocco interno)

Passare una chiusura Passare dinamicamente sia il legame col codice della funzione, che il suo ambiente non locale, cioè una chiusura $\langle code, env \rangle$.

Quindi allochiamo (come sempre) RdA, prendiamo il puntatore di catena statica dalla chiusura.

Come li implementiamo?

- Scope statico
 - (SEMPRE) Deep: implementato con chiusure
 - Shallow: non si fa, ma non fa differenza con deep
- Scope dinamico
 - Deep: usa necessariamente qualche forma di chiusura per “congelare” uno scope da riattivare più tardi
 - Shallow: per accedere ad x risali la pila (A-list, CRT)

4.3 Upward funarg problem

Alcuni linguaggi (eg funzionali) permettono di restituire una funzione. Se la funzione ha variabili locali queste devono sopravvivere indipendentemente dalla struttura a pila: hanno vita indefinitamente lunga.

```

{int x = 1;
void->int F () {
    int g () {
        return x+1;
    }
    return g;
}
void->int gg = F();
int z = gg();
}

```

F ritorna una chiusura.

La pila (LIFO) non va più bene, come implementiamo la "pila" corretta?

- non deallocare esplicitamente
- record di attivazione sullo heap
- le catene statica e dinamica collegano i record
- invoca il garbage collector quando necessario

4.4 Eccezioni

Terminare parte di una computazione quando richiesto e gestire l'eccezione se sorta.

• La funzione **average** calcola la media di un vettore; se il vettore è vuoto, solleva eccezione

```
class EmptyExcp extends Throwable {int x=0;};

int average(int[] V) throws EmptyExcp() {
    if (length(V)==0) throw new EmptyExcp();
    else {int s=0; for (int i=0, i<length(V), i++) s=s+V[i];}
    return s/length(V);
};

try {
    ...
    average(W);
    ...
}
catch (EmptyExcp e) {write('Array_vuoto'); }
```

solleva l'eccezione

eccezione

intrappola e gestisce l'eccezione

gestore (handler) dell'eccezione

Il gestore è legato in modo statico al blocco di codice protetto, l'esecuzione del gestore rimpiazza la parte di blocco che doveva essere ancora eseguita.

Possiamo anche utilizzare le eccezioni per rendere più efficiente un codice:

Problema: Calcolare il prodotto tra tutti i nodi di un albero

- NON OTTIMIZZATA:

```
int mul(T tree){
    if (tree == null) return 1;
    return tree.value * mul(tree.L) * mul(tree.R);
}
```

- OTTIMIZZATA:

```
int mulAus(T tree){
    if (tree == null) return 1;
    if (tree.value == 0) {
        throw new Zero();
    }
    return tree.value * mul(tree.L) * mul(tree.R);
}
```

```

    }

    int mul(T tree){
        try{
            return mulAus(tree)
        } catch (Zero e){
            return 0;
        }
    }
}

```

Come implementiamo le eccezioni? Il compilatore genera una *tabella delle eccezioni* che, per ogni blocco protetto (`try`), contiene una coppia del tipo

$$\langle block_addr, handler_addr \rangle$$

(dove in realtà il blocco è identificato da un intervallo di indirizzi).

La tabella è ordinata staticamente rispetto a *block_addr*.

Quando viene sollevata un'eccezione, il runtime prende l'indirizzo corrente di esecuzione (Program Counter) e cerca nella tabella il blocco che lo contiene, tipicamente tramite *ricerca binaria*.

Una volta individuata la voce corrispondente, il controllo viene trasferito all'indirizzo del gestore (*handler_addr*), dopo aver effettuato lo *stack unwinding*. Se il gestore rilancia l'eccezione, il processo viene ripetuto nel frame chiamante, continuando la ricerca di un gestore compatibile. Se non lo trova il programma finisce senza gestire l'eccezione.

5 Introduzione ai Tipi

Il termine **sistema di tipi** (o teoria dei tipi) si riferisce a un ampio campo di studi in logica, matematica e filosofia. I primi ricercatori hanno formalizzato i sistemi di tipi in questo senso all'inizio del 1900 per evitare i **paradossi logici**, come il paradosso di Russell, che minacciavano i fondamenti della matematica.

In informatica esistono due grandi branche di ricerca sui sistemi di tipi:

- quella più **pratica** riguarda le applicazioni ai linguaggi di programmazione
- quella più **astratta** si concentra sulle connessioni con le varietà della logica

Supporto all'Organizzazione Concettuale I tipi possono aiutare a disciplinare l'organizzazione concettuale dei programmi. I tipi consentono ai programmatori di esprimere la differenza tra le entità che formano la soluzione di un determinato problema. L'uso di tipi distinti è uno strumento di **documentazione** e di **progettazione**.

Supporto all'Astrazione Una particolare declinazione del supporto all'organizzazione concettuale offerto dai tipi è quella di dare supporto concreto ai sistemi di moduli nei linguaggi dove i moduli “impacchettano” e legano insieme diverse unità software. L'esempio tipico di questo tipo di utilizzo dei tipi sono le **interfacce**, che **associano un tipo** (per esempio, l'intero) **ad operazioni** che si possono applicare su di esso (per esempio, +, -, %, conversioni).

Supporto alla Correttezza Il vantaggio più famoso dei tipi è il **type-checking**, cioè la possibilità di usare i tipi per rilevare errori di programmazione. I tipi aiutano molto anche il **refactoring**. In generale, la “safety” si riferisce alla capacità di un linguaggio di **garantire l’integrità delle sue astrazioni**.

Supporto all’Implementazione I tipi possono contribuire a migliorare l’efficienza dei programmi; nei linguaggi safe, i tipi aiutano a migliorare l’efficienza eliminando molti dei controlli dinamici per garantire la sicurezza.

Dynamic vs Static Typing

- Un linguaggio è “tipato staticamente” (statically typed) se possiamo controllare i tipi sul testo del programma, senza doverlo eseguire (compilazione)
- Si parla di linguaggi “tipati dinamicamente” (dynamically typed) quando il controllo dei tipi avviene durante l’esecuzione del programma

Manifest vs Inferred typing

La tipizzazione statica o dinamica riguarda il momento in cui un linguaggio (interpretato o compilato) esegue il controllo dei tipi.

- Un linguaggio con manifest typing richiede che il programmatore annoti tutti i tipi di variabili e operazioni. Un linguaggio con inferred typing non ha bisogno di annotazioni, in quanto dispone di algoritmi che deducono i tipi dal contesto (dichiarazioni, operazioni).
- Mentre il tipaggio statico e quello dinamico lasciano poco spazio (utile) all’ibridazione, il tipaggio manifesto e inferito sono uno spettro determinato da fattori ergonomici e algoritmici.

6 Tipi Base e l’Algebra dei Tipi

Ogni linguaggio di programmazione ha un proprio sistema di tipi, cioè le informazioni e le regole che governano i tipi e i loro valori, chiamati abitanti (inhabitants) del tipo.

1. un insieme di **tipi di base**
2. meccanismi per **definire nuovi tipi**
3. meccanismi di computazione sui tipi:
 - **regole di equivalenza**: che specificano quando due tipi corrispondono allo stesso tipo
 - **regole di compatibilità**: che specificano quando si può usare un tipo al posto di un altro
 - **regole/tecniche di inferenza**: specificano come assegnare un tipo a un’espressione, a partire dalle informazioni sui suoi componenti;
4. la definizione sul controllo dei vincoli di tipo statico o dinamico.

Tipo Unit (e Void) Il tipo più elementare è quello che contiene un solo elemento (l'insieme è un singoletto). L'unico abitante del tipo Unit è l'unità singoletto (rappresentato anche come ()) e di solito è associato a operazioni il cui tipo di ritorno non è utilizzabile. Un concetto simile all'unità col tipo void che, tuttavia, presenta alcune differenze con Unit. Per esempio, mentre possiamo passare l'unità come argomento delle operazioni, non possiamo ottenere né passare un void.

Tipi Booleani I booleani indicano il tipo di valori logici e di solito comprendono **true** e **false**. Le operazioni logiche sono congiunzione (&), la disgiunzione (|), la negazione (!), l'uguaglianza (==), lo XOR (^).

Tipi Carattere I caratteri (chars) indicano tutti i caratteri di un determinato (e definito a livello linguistico) insieme di simboli. Include valori (ASCII/UNICODE) e operazioni dipendenti da linguaggio come uguaglianza e confronti.

Tipi Interi Indicano un intervallo (finito) di numeri interi con o senza segno. Supporta operazioni di uguaglianza, confronti e operazioni aritmetiche (+, -, *, %).

Tipi Reali I floating point indicano un intervallo (limitato) di numeri reali a virgola fissa/mobile. Possono non essere distribuiti ugualmente in tutto l'intervallo!

Enumeration types Un tipo di enumerazione consiste in un insieme finito di costanti, ciascuna caratterizzata da un proprio nome.

```
enum Color {  
    Red,  
    Green,  
    Blue  
}
```

Non tutti i linguaggi integrano gli enum in **modo sicuro**, in C per esempio è zucchero sintattico su interi

```
typedef Color {  
    Red = 0,  
    Green = 1,  
    Blue = 2  
}
```

questo impedisce di verificare la correttezza e di distinguerli da *int*.

Tipi Array Un tipo array denota un insieme di elementi di un certo tipo, ciascuno indicizzato da almeno una **chiave identificativa** di un certo tipo. Per $n + 1$ elementi si indica l'intervallo $[0, n]$, e si lascia all'utente indicare il tipo degli elementi.

Altre versioni di array come *mappe* (map) o array associativi permettono all'utente di fissare sia il tipo per la chiave che per gli elementi.

```
map<string, int> Database; // Hashmap che dato un nome ritorna un int
```

Come operazioni abbiamo la **selezione** tramite l'indice, assegnazione, confronti e operazioni aritmetiche. I linguaggi safe verificano che gli accessi siano nei limiti dell'array.

Tipi Riferimento Un riferimenti da accesso indiretto ad un altro valore. Le operazioni sono **creazione**, controllo di uguaglianza e **deferenziazione** (accesso al dato referenziato). Sono essenziali per linguaggi a basso livello. Vengono fortemente applicati attraverso i puntatori

```
float pi = 3.14;
float *p = NULL;
p = &pi; // Il puntatore p punta alla posizione che contiene pi.
*p+=1; //fa diventare pi=4.14 deferenziando p
```

Bisogna stare attenti alla **deallocazione implicita**

```
int *p = malloc(sizeof(int));
*p = 5;
p = NULL; //rendiamo inaccessibile la locazione di malloc precedente!
```

Questa porzione di memoria “non liberata” può crescere nel tempo (finché il programma viene eseguito) e può incorrere in un fenomeno chiamato “memory leak”. Questo problema può essere risolto via il *garbage collector*.

Nei linguaggi con riferimenti troviamo un **operatore di deallocazione esplicito**:

```
int *p = malloc(sizeof(int));
*p = 5;
free(p) //deallochiamo la sezione di memoria di malloc
p = NULL;
```

Tipi Prodotto Array, set e puntatori sono esempi di tipi composti che "prendono come parametro" un tipo. Quando si combinano due o più tipi in una struttura fissa, si parla di tipi prodotto. I tipi di prodotto più comuni sono coppie (pair), tuple, record e varianti. Per convenzione il vuoto è unita' (Unit).

Tipi Record I record interpretano i tipi prodotto sostituendo la definizione posizionale dei componenti con la loro identificazione mediante etichette (distinte).

```
struct Person{
    char *name;
    int age;
}
```

Pattern Matching Tecnica per i record per semplificare i dati strutturati, attraverso costrutti che rendono semplice destrutturarli.

```
let evaR = PersonR{ name, age: match age {
    1..=10 => [ 'K', 'i', 'd', '' ],
    11..=20 => [ 'T', 'e', 'e', 'n' ],
    _ => [ 'O', 'l', 'd', '' ]
}}
```

Tipi Ricorsivi I tipo ricorsivi (concetto) sono strutture dati (liste, alberi, ecc.) che possiamo modificare dinamicamente.

```
struct List{
    int val;
    List* next;
}
```

Tipi Somma Quando un tipo e' a variabile di tipo composta del tipo

```
x := int ∪ char
x = {5, j, 17, i}
```

Tipi Funzione I linguaggi di alto livello supportano spesso la definizione di funzioni (o procedure), tuttavia solo alcuni denotano il tipo di funzioni.

$$T_f : \begin{matrix} f(T\ p)\{ \\ \vdots \\ \} \end{matrix}$$

Puo' essere definita come

$$f : T_f \rightarrow T$$

7 Tipi Polimorfi

Se un sistema di tipi non ci permette di esprimere che alcuni tipi accettano altri tipi come parametri, chiamiamo il sistema di tipi **monomorfico**.

Ipotizziamo di volere definire una funzione per trovare il max per *int* e *float*, in un sistema monomorfico definiremo:

$$\begin{matrix} \max(int, int) \rightarrow int \\ \max(float, float) \rightarrow float \\ \vdots \end{matrix}$$

Invece in un sistema a **polimorfismo** di tipo consente agli sviluppatori di specificare un insieme di operazioni su un dato tipo parametrico che **astrae da alcuni dettagli dell'istanziatura concreta del tipo**.

In generale, esistono tre tipi di polimorfismo:

- **ad-hoc (sovraccarico/overloading)**: sfrutta la capacità del compilatore/runtime del linguaggio di distinguere dal contesto di chiamata definizioni alternative di operazioni con lo stesso nome.

```
int max(int i, int j){
    return i > j ? i : j;
}
float max(float i, float j){
    return i > j ? i : j;
}
```

- **di sottotipo (subtyping)**: si basa su una relazione di sottoinsieme, per esempio

$$S <: T$$

si dice che S e' sottotipo di T , ovvero S e' piu' specifico di T , possiamo utilizzare S in qualsiasi posto che necessiti un T .

Abbiamo due tipi di sottotipaggio, consideriamo $Dog <: Animal$:

- **sottotipaggio in larghezza**: i record dei sottotipi aggiungono campi

```
type Animal:{name:string}
type Dog:{name:string , back:string}
```

- **sottosipaggio in profondita'**: sostituiamo campo con sottotipi

```
type AnimalHouse:{tenant: Animal}
type DogHouse:{tenant: Dog}
```

Nel caso della funzione max per *int* e *float* definiamo

$$int <: float$$

```
max(float i , float j) -> float {
  ...
}
```

- **parametrico (universale)**: non si possono fare ipotesi sulla forma dei parametri del tipo

```
//Definiamo i tipi
int <: Comparable
float <: Comparable

type T extends Comparable;

T max(T x, T y) -> T { //Definiamo la funzione
  ...
}
```

Quindi abbiamo definito

$$\text{max} := T \times T \rightarrow T \quad (1)$$

8 Exceptions

A seconda delle istruzioni che vogliamo eseguire, potremmo aver bisogno di programmare operazioni che **falliscono**; oltre a poter **esprimere** queste situazioni nei programmi, dovremmo (dobbiamo) anche trovare il modo di **strutturarle**.

L'idea alla base delle eccezioni è che una condizione eccezionale (un raro evento di fallimento) causa il trasferimento diretto **del controllo a un gestore di eccezioni** definito in qualche punto dello stack di chiamate del programma.

```
try{
  x = 1 / 0
} catch(ArithmeticException exception){
  print("Non si divide per 0!")
}
```

Il blocco catch accetta un argomento: l'eccezione che si aspetta di intercettare.

Il codice nel blocco **try** non viene eseguito "atomicamente". Infatti, il codice nel blocco catch sostituisce il codice nel blocco try solo a partire dall'istruzione che solleva (**throw**).

Concentrandosi sulle Exception, Java obbliga gli sviluppatori a gestirle esplicitamente in due modi possibili: o dichiarando che un'operazione lancia un'eccezione o gestendo l'eccezione lanciata all'interno dell'operazione.

```

void o() throws MyException{
    throw new MyException();
}

void o(){
    try {
        throw new MyException();
    } catch ( MyException e ){
        ...
    }
}

```

9 Sicurezza della memoria: Garbage collection e Borrow-checking

I riferimenti quando fanno riferimento a memoria deallocata/reallocata diventano **dangling** e posso portare **undefined behaviour** (comportamenti inaspettati).

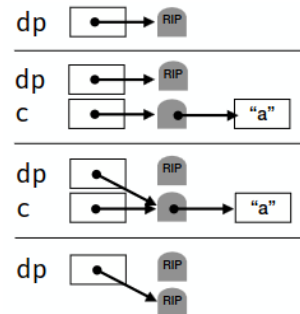
9.1 Tombstones

Un modo per gestire i punattori e' con le tombstones, ovvero si associa ad ogni allocazione una cella di memoria extra. All'inizializzaione la tombstone contiene l'indirizzo dell'elemento e il puntatore punta alla tombstones. La deferenziazione funziona prima alla tombstones e poi all'oggetto.

```

{
    char *dp = NULL;
    {
        char c = "a";
        dp = &c;
    }
}

```



Il costo maggiore e' i ltempo di deallocazione perche' e' duplicato e il costo di memoria occupata dalla tombstone.

9.2 Locks and Keys

I lock-and-keys sono un'alternativa alle tombstones, ma solo per i puntatori allo heap.

Ogni volta che allochiamo un oggetto sull'heap, lo **associamo a un "lock"** costituito da una parola casuale in memoria.

$obj \rightarrow \langle value, key \rangle$

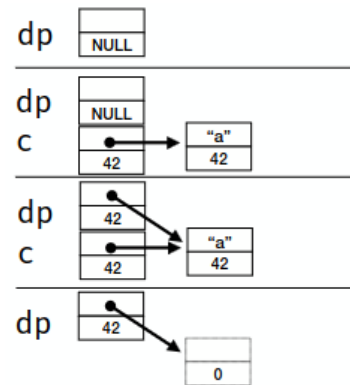
Un puntatore consiste in una coppia: l'indirizzo effettivo e una "chiave" (key), cioè una parola in memoria inizializzata al valore del blocco dell'oggetto puntato.

Ogni volta che si dereferenzia un puntatore, si controlla che la chiave possa "aprire" il lock, (cioè che le due parole coincidano).

```

{
    char *dp = NULL;
    {
        char c = "a";
        dp = &c;
    }
}

```



Locks-and-keys ha un costo significativo. In termini di spazio, costano anche più delle tombstones, poiché è necessaria una parola aggiuntiva per ogni puntatore. Oltre al costo di equivalenza per creazione/assegnazione/deferenziazione.

9.3 Garbage Collection

Meccanismo per gestire automaticamente la deallocazione degli oggetti non più utilizzati. Da un punto di vista logico, le operazioni di un garbage collector comprendono:

- **Garbage detection:** rilevare se gli oggetti in memoria sono in uso o meno
- **Garbage collection:** rilasciare la memoria occupata dagli oggetti non utilizzati

Vediamo le tecniche più comuni di garbage collection.

9.3.1 Reference Count

La tecnica dei contatori di riferimenti segue questa definizione ed è probabilmente il modo più elementare per realizzare un garbage collector.

Quando si alloca un oggetto nello heap, il runtime inizializza anche un contatore di riferimenti (un intero, inaccessibile al programmatore) di quell'oggetto e si assicura di mantenere il contatore di ogni oggetto sincronizzato con il numero di puntatori attivi a quell'oggetto. Al momento della creazione il contatore è 1. Il runtime aumenta il contatore ogni volta che assegniamo il suo puntatore a una variabile puntatore e lo diminuisce ogni volta che perde una variabile puntatore.

Quando un contatore di riferimento raggiunge lo 0, il suo oggetto diventa garbage e possiamo deallocarlo dalla memoria. Oltretutto prima di segnare un oggetto come garbage dobbiamo decrementare tutti i suoi puntatori.

Algorithm 1 Release con reference counting

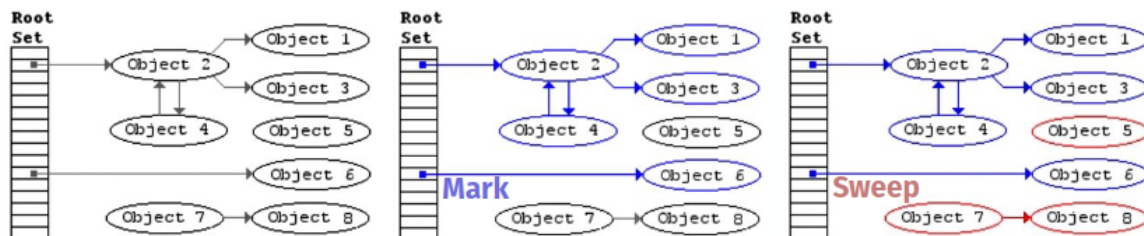
```
procedure RELEASE(obj)
  if obj = null then
    return
  end if
  if obj.counter > 0 then
    return
  end if
  for all v ∈ obj.variables do
    if v.type = pointer ∧ v.value ≠ null then
      v.value.counter ← v.value.counter - 1
      if v.value.counter = 0 then
        RELEASE(v.value)
      end if
    end if
  end for
  DEALLOCATE(obj)
end procedure
```

Nota: Essendo una visita ricorsiva non e' possibile deallocare strutture ricorsive (Liste circolari, ecc).

9.3.2 Mark and Sweep

La tecnica mark-and-sweep prende il nome dal modo in cui esegue detection e collection.

- **Mark:** indica che la detection utilizza contrassegni per identificare il garbage, seguendo due fasi.
 - Attraversa l'heap e contrassegna tutti gli oggetti come garbage
 - Attraversa lo stack, segue i puntatori attivi agli oggetti nello heap e rimuove i contrassegni precedenti da quelli visitati (ricorsivamente)
- **Sweep:** è una visita piatta dell'heap per raccogliere tutti gli oggetti marcati come garbage



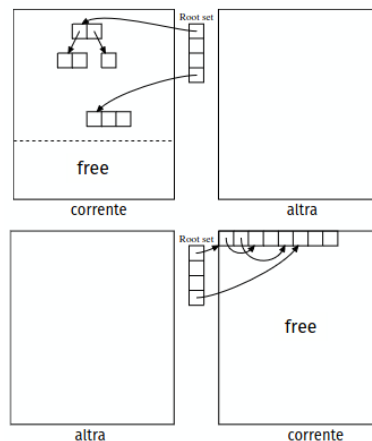
Una differenza con il reference counting e' che la memoria quando diventa garbage non viene liberata istantaneamente ma solo quando il runtime lo ritiene utile.

Generalmente mark and sweep e' piu' efficiente e facile da implementare, ma rallenta molto le prestazioni e reattività

9.3.3 Stop and Copy

Evoluzione del mark and sweep che consente di ottenere la pulizia dello heap eliminando sia la prima fase della marcatura che lo sweep.

La tecnica funziona dividendo l'heap in due regioni di uguale dimensione. Tutte le allocazioni avvengono nella metà "corrente", mentre l'altra è vuota. Quando la metà corrente è quasi piena il garbage collector esplora la metà "corrente", a partire dall'insieme delle radici nello stack, e copia ogni oggetto raggiungibile in modo contiguo nell'altra metà. Quando il garbage collector termina l'esplorazione, scambia i puntatori dalla metà "corrente" e all'altra.



9.3.4 Borrow checking

Tecnica (presente in Rust) di sicurezza nel quale se i programmi vengono compilati correttamente sappiamo che è privo di errori di puntatori dangling e wild, double free, ecc.

La tecnica si basa sulla definizione, nel linguaggio, di una proprietà di possesso (*ownership*), dove ogni valore del programma ha un *unico proprietario* (una variabile) che ne determina la durata (*lifetime*) in memoria.

Ownership Il concetto di ownership che abbiamo visto finora è piuttosto semplice: si tratta essenzialmente di un albero in cui, deallocando la radice, sappiamo di poter deallocare tutti i suoi sottonodi. Questo consente anche le strutture annidate (non circolari).

- Possiamo **passare (move) la ownership** dei valori da un proprietario ad un altro
 - In Rust, per la maggior parte dei tipi (a parte alcune eccezioni, discusse in seguito), operazioni come l'assegnazione di un valore a una variabile, il passaggio di un valore a una funzione o la restituzione di un valore non eseguono alcuna copia, ma spostano la proprietà del valore.

```
let x = vec![10, 20, 30];
if false {
    let a = x;
} else {
    let c = 2;
}
let c = x;
```

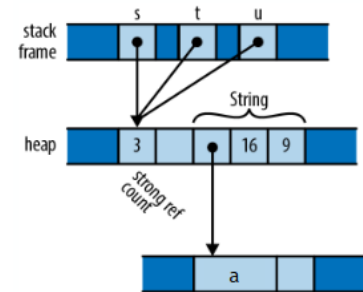
Questo codice non viene compilato, infatti il principio generale è che, se è possibile che una variabile abbia spostato la proprietà del proprio valore (e non è certo che

da allora le sia stato assegnato un nuovo valore) consideriamo la variabile non inizializzata.

Per alcuni tipi di variabili (int, float, char, bool, fix. size array) e' abbastanza efficiente (e sicuro) la copia rispetto che spostarli. Al contrario per *string*, *dynamic array*, ecc essendo allocato in heap non e' conveniente.

- Possiamo dare maggiore libertà dalle regole di ownership ad alcuni tipi semplici, come gli interi, i floats e i caratteri
- Possiamo fornire tipi speciali di puntatore con conteggio dei riferimenti (Rc e Arc), che consentono ai valori di avere più proprietari, con alcune restrizioni
 - Se abbiamo valori a cui serve condividere tra proprietari (per esempio per var. immutabili) possiamo definire
 1. **Reference count** (Rc<T>)
 2. **Atomic reference counte** (Arc<T>)

```
let s: Rc<String> = Rc::new("a".
    to_string());
let t: Rc<String> = s.clone();
let u: Rc<String> = s.clone();
```



Ognuno dei tre puntatori *Rc<String>* si riferisce allo stesso blocco di memoria.

- Possiamo permettere ai proprietari di "*prendere in prestito un riferimento*" (**borrow**) a un valore, a patto di limitare i riferimenti a puntatori non proprietari con durata limitata

9.4 Lifetimes

Il compilatore di Rust convalida i "prestiti" ragionando sulle lifetime delle variabili; essenzialmente si assicura che nessun riferimento sopravviva al proprietario da cui prende in prestito il valore.

```
let x: &str;
{
    let y = "a"; //y e' definita solo dentro lo scope
    x = &y; //errore perche fuori dallo scope y e' deallocata
}
println!( "{}", x );
```

Le interazioni tra le variabili (prestiti) non modificano i lifetime delle variabili, ma impongono vincoli tra queste che il compilatore deve controllare.

10 Tipi di Dato Astratto e Object Orientation